

Traffic Engineering adattivo in reti SDN

Rerouting dinamico basato sul monitoraggio delle statistiche OpenFlow

Fabio Liguoro - Matricola DE9000067

1. Obiettivo

Il progetto realizza una **SDN Application** che esegue Traffic Engineering in modo adattivo: il controller osserva in tempo reale il carico dei collegamenti e, quando il percorso primario si satura, riconfigura il data plane spostando un flusso sul percorso alternativo. La riconfigurazione avviene senza interrompere il traffico e viene revocata automaticamente quando la domanda torna a essere sostenibile su un solo percorso.

L'obiettivo non è soltanto ottenere un sistema funzionante, ma misurare il beneficio del Traffic Engineering, infatti, l'esperimento viene ripetuto in condizioni identiche con la politica di controllo disattivata, così da poter confrontare quantitativamente i due scenari.

2. Inquadramento teorico

2.1 Perché SDN semplifica il Traffic Engineering

Nell'approccio tradizionale MPLS il percorso vincolato è calcolato dall'Ingress LER mediante l'algoritmo CSPF e successivamente instaurato tramite segnalazione RSVP-TE lungo tutti i nodi del cammino. Questo schema presenta due limiti strutturali: il calcolo si basa sulla vista locale del nodo di ingresso, e ogni modifica richiede un protocollo di segnalazione distribuito che coinvolge l'intero percorso.

La separazione fra Control Plane e Data Plane propria di SDN elimina entrambi i problemi. Il controller possiede una vista globale e centralizzata dello stato dei collegamenti; quindi, il calcolo del nuovo percorso non richiede alcuna approssimazione locale; e la sua attuazione si riduce all'invio di messaggi FlowMod ai soli switch interessati. Nel sistema realizzato uno spostamento di percorso richiede due soli messaggi.

2.2 Policy-Based Network Management

La logica di controllo è espressa nella forma canonica del PBNM, IF <network condition> THEN <enforce configuration action>:

```
IF (utilizzo aggregato > soglia alta) THEN sposta il flusso sul percorso secondario
IF (utilizzo aggregato < soglia bassa) THEN riconsolido il flusso sul percorso primario
```

2.3 Modello di monitoraggio a cinque livelli

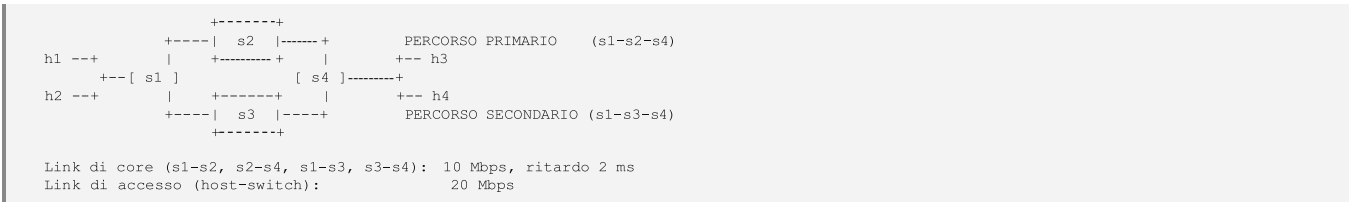
Livello	Componente
Collection	Interrogazione periodica dei contatori di porta degli switch (OFPPortStatsRequest) ogni 2 s
Representation	Conversione dei contatori cumulativi in throughput (bit/s) e normalizzazione rispetto alla capacità del collegamento
Report	Trasporto dei campioni dallo switch al controller sul canale di controllo OpenFlow
Analysis	Confronto con le soglie, isteresi e conferma della condizione su più campioni consecutivi
Presentation	Log su console e registrazione su file CSV, da cui sono generati i grafici della Sezione 5.3

Il monitoraggio è di tipo passivo: non viene iniettato traffico di prova nella rete, ma si leggono i contatori che gli switch aggiornano comunque per ogni pacchetto inoltrato. Il costo sul canale di controllo è di un solo scambio richiesta/risposta ogni due secondi.

3. Architettura del sistema

3.1 Topologia

La topologia, emulata in **Mininet** con switch **Open vSwitch**, è a diamante: due percorsi disgiunti collegano lo switch di ingresso s1 a quello di uscita s4.



I collegamenti di accesso hanno capacità doppia rispetto a quelli interni: in questo modo l'unico punto in cui può formarsi congestione è il percorso di core, che è l'oggetto dello studio. Le capacità sono imposte tramite TCLink, che applica i limiti attraverso la disciplina di coda HTB del kernel Linux.

I numeri di porta OpenFlow sono fissati esplicitamente nello script della topologia e non lasciati all'assegnazione automatica: le flow entry specificano la porta di uscita; quindi, la numerazione deve essere deterministica e indipendente dall'ordine di creazione delle interfacce.

Switch	Porta 1	Porta 2	Porta 3	Porta 4
s1	h1	h2	s2 (primario)	s3 (secondario)
s2	s1	s4	-	-
s3	s1	s4	-	-
s4	h3	h4	s2 (primario)	s3 (secondario)

3.2 Assenza di flooding

La topologia contiene un ciclo fisico (s1-s2-s4-s3-s1) e gli switch non eseguono lo Spanning Tree Protocol. Un learning switch classico, non conoscendo la posizione di un indirizzo MAC, ricorrerebbe al flooding e i pacchetti circolerebbero indefinitamente nell'anello (*broadcast storm*). Il progetto evita del tutto la situazione con due accorgimenti:

- tutte le flow entry sono installate in modalità **proattiva** e realizzano un inoltra esplicito basato sugli indirizzi IP, mai un flooding;
- gli host sono configurati con **entry ARP statiche**, così che nessuna risoluzione di indirizzo avvenga a runtime e nessun pacchetto

broadcast entri in rete. IPv6 è disabilitato per la stessa ragione.

4. Il controller

Il controller è una applicazione **Ryu** che dialoga con gli switch tramite **OpenFlow 1.3**

Livello	Che cosa avviene nel sistema realizzato
Collection	Ogni 2 s il controller invia <code>OFPPortStatsRequest</code> a s1 e lo switch legge i contatori cumulativi <code>tx_bytes</code> delle porte 3 e 4. Monitoraggio passivo: nessun traffico di prova viene iniettato in rete.
Report	La risposta <code>OFPPortStatsReply</code> raggiunge il controller attraverso il canale di controllo OpenFlow. Il periodo di polling regola qui il compromesso fra precisione temporale della misura e banda consumata sul canale.
Representation	I contatori cumulativi diventano una velocità: $\text{throughput} = (\Delta \text{tx_bytes} \times 8) / \Delta t$. Il valore viene poi normalizzato: $U = (\text{primario} + \text{secondario}) / \text{capacità di un link}$.
Analysis	U viene confrontato con le due soglie -> se supera 0,80 per due campioni consecutivi si passa allo stato SPLIT, se scende sotto 0,30 si torna a CONSOLIDATED.
Attuazione	Due messaggi <code>FlowMod</code> di tipo <code>OFPPC_MODIFY_STRICT</code> verso s1 e s4 modificano la porta di uscita della entry del flusso elastico. Il data plane è riconfigurato senza interrompere il traffico, e il ciclo riprende dal passo 1.
Presentation	In parallelo a ogni campione, i valori vengono scritti sul log di console e sul file CSV da cui sono generati i grafici della Sezione 5.3.

4.1 Programmazione proattiva delle flow table

Alla ricezione del messaggio *Features Reply*, cioè al termine dell'handshake, il controller popola immediatamente la flow table dello switch, prima che arrivi qualunque pacchetto di traffico. Le entry sono organizzate su tre livelli di priorità:

Priorità	Match Fields	Ruolo
20	eth_type, ipv4_src, ipv4_dst	Instradamento specifico per flusso, sugli switch di bordo s1 e s4
10	eth_type, ipv4_dst	Consegna locale agli host e transito sui nodi s2 e s3
0	(wildcard totale)	Table-miss: invio al controller dei soli primi 64 byte, per diagnostica

Le regole che identificano un flusso attraverso la coppia sorgente-destinazione devono prevalere su quelle che osservano la sola destinazione, altrimenti un pacchetto in transito verrebbe erroneamente consegnato in locale.

Un aspetto rilevante è che **entrambi i percorsi sono programmati fin dall'avvio**, incluso quello secondario che inizialmente non trasporta traffico; perciò, i nodi di transito non devono essere riconfigurati al momento dello spostamento. Il rerouting si riduce a cambiare la porta di uscita sui due switch di bordo, con il vantaggio di non introdurre alcun ritardo di provisioning.

4.2 Misura del carico e osservazioni

I contatori OpenFlow sono **cumulativi**. Il throughput è quindi ottenuto per differenza fra due letture successive, dividendo per l'intervallo effettivamente misurato e non per il periodo nominale di polling, poiché lo scheduling non è perfettamente regolare e un errore sull'intervallo si tradurrebbe in un errore proporzionale sulla stima.

```
throughput = (tx_bytes(t2) - tx_bytes(t1)) * 8 / (t2 - t1)
```

U rappresenta quanti percorsi sono necessari per trasportare il traffico attualmente presente in rete. Ci fa capire che, dopo lo spostamento, il collegamento primario resta comunque carico; perciò, una condizione di rientro basata sul solo primario non si verificherebbe mai e il traffico non tornerebbe mai a consolidarsi.

```
U = (banda sul primario + banda sul secondario) / capacità di un link
```

Le due soglie (0,80 per l'attivazione e 0,30 per il rientro) servono perché con un'unica soglia il sistema oscillerebbe indefinitamente. Spostato il flusso, il collegamento si scarica, la condizione di rientro diventa immediatamente vera, il flusso torna indietro e il collegamento si risatura. La banda morta fra le due soglie rende il comportamento stabile.

A ciò si aggiunge la richiesta che la condizione sia confermata da **due campioni consecutivi** perché bisogna filtrare i picchi istantanei visto che un singolo campione al di sotto della soglia azzererebbe il conteggio.

4.3 L'azione di riconfigurazione

Lo spostamento richiede due soli messaggi `FlowMod`, uno per ciascuna direzione del flusso, inviati rispettivamente a s1 e s4. Viene impiegato il comando `OFPPC_MODIFY_STRICT` anziché una coppia *delete + add*, per due ragioni:

- MODIFY sostituisce le sole azioni lasciando la entry in tabella, quindi non esiste alcun istante in cui il pacchetto non trovi corrispondenza e finisca in table-miss;
- la variante STRICT impone che Match Fields e priorità coincidano esattamente con quelli della entry esistente, evitando di modificare inavvertitamente altre regole sovrapposte.

5. Scenario sperimentale e risultati

5.1 Scelta del traffico

Il traffico è generato con **iperf3** in modalità **UDP a bitrate costante**. Non uso TCP perché il controllo di congestione porta a occupare tutta la banda disponibile. Il collegamento risulterebbe saturo in qualsiasi condizione e il controller non potrebbe distinguere un carico modesto da una congestione reale. Con UDP a bitrate fisso la domanda di traffico è una grandezza imposta sperimentalmente, il che consente di osservare l'intero ciclo della politica di controllo.

Istante	Traffico offerto	Comportamento atteso
t = 0 s	Flusso A (h1→h3), 2 Mbps	U ≈ 0,20: sotto entrambe le soglie, nessuna azione
t = 20 s	+ Flusso B (h2→h4), 8 Mbps	10 Mbps richiesti su un link da 10: U → 1,00, superata la soglia alta, rerouting
t = 50 s	Termina il flusso B	U ≈ 0,20: sotto la soglia bassa, rientro sul primario

Il flusso A resta ancorato al percorso primario e funge da traffico di fondo; il flusso B è quello *elastico*, oggetto dello spostamento. Lo

scenario di riferimento (politica disattivata) è ottenuto portando la soglia alta a un valore irraggiungibile, quindi il controller mi sura esattamente come prima ma non interviene mai.

5.2 Verifica sul data plane

Il log del controller documenta la catena che porta dalla misura all'attuazione. I due campioni consecutivi sopra la soglia alta confermano la condizione di congestione, dopodichè vengono inviati i messaggi FlowMod:

```
[t= 30.0s] primario= 9.91 Mbps secondario= 0.00 Mbps U= 99.1%
stato=CONSOLIDATED

soglia alta superata (1/2)

[t= 32.0s] primario= 9.75 Mbps secondario= 0.00 Mbps U= 97.5%
stato=CONSOLIDATED

soglia alta superata (2/2)

>>> REROUTE: flusso h2<->h4 spostato su SECONDARIO (s1-s3-s4)

[t= 34.0s] primario= 5.67 Mbps secondario= 9.30 Mbps U=
149.6% stato=SPLIT

[t= 36.0s] primario= 2.07 Mbps secondario= 9.18 Mbps U=
112.5% stato=SPLIT
```

L'ispezione diretta della flow table di s1 conferma l'effetto del messaggio FlowMod sulla entry del flusso elastico:

```
prima: priority=20,ip,nw_src=10.0.0.2,nw_dst=10.0.0.4
actions=output:"s1-eth3"

dopo: priority=20,ip,nw_src=10.0.0.2,nw_dst=10.0.0.4
actions=output:"s1-eth4"
```

Nello scenario di riferimento la medesima entry rimane invariata per tutta la durata dell'esperimento, a riprova che lo spostamento è opera esclusiva della politica di controllo.

5.3 Risultati quantitativi

Flusso elastico h2→h4 (8 Mbps offerti)	Senza Traffic Engineering	Con Traffic Engineering	Variazione
Throughput medio ricevuto	7,24 Mbps	8,98 Mbps	+24,0 %
Datagrammi persi	4514 / 28125 (16 %)	0 / 28124 (0 %)	azzerata
Jitter finale	1,233 ms	0,195 ms	-84 %
Datagrammi fuori ordine	1	594	costo transitorio

Uso delle risorse di rete (30 s di carico)	Senza Traffic Engineering	Con Traffic Engineering
Traffico sul percorso primario	8,84 Mbps	3,34 Mbps
Traffico sul percorso secondario	0,00 Mbps	7,88 Mbps
Totale trasportato	8,84 Mbps	11,22 Mbps
Uso della capacità disponibile (2 × 10 Mbps)	44,2 %	56,1 %

Il dato più significativo è il valore nullo sul percorso secondario nello scenario di riferimento: **un intero collegamento resta inutilizzato mentre l'altro è saturo al 100 % e scarta il 16 % dei pacchetti**. La capacità era disponibile, ma manca un meccanismo in grado di sfruttarla.

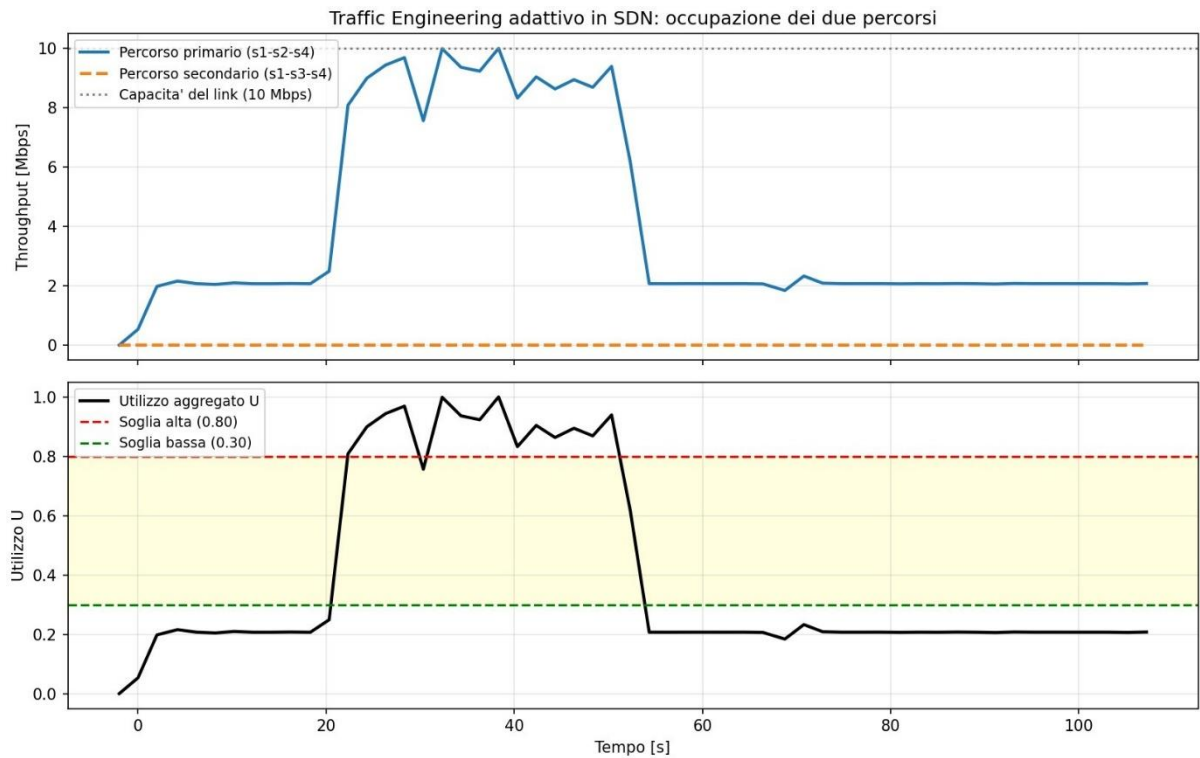


Figura 1 - Traffic Engineering disattivato. Il percorso secondario resta a zero per l'intera durata; l'utilizzo del primario tocca il 100 %

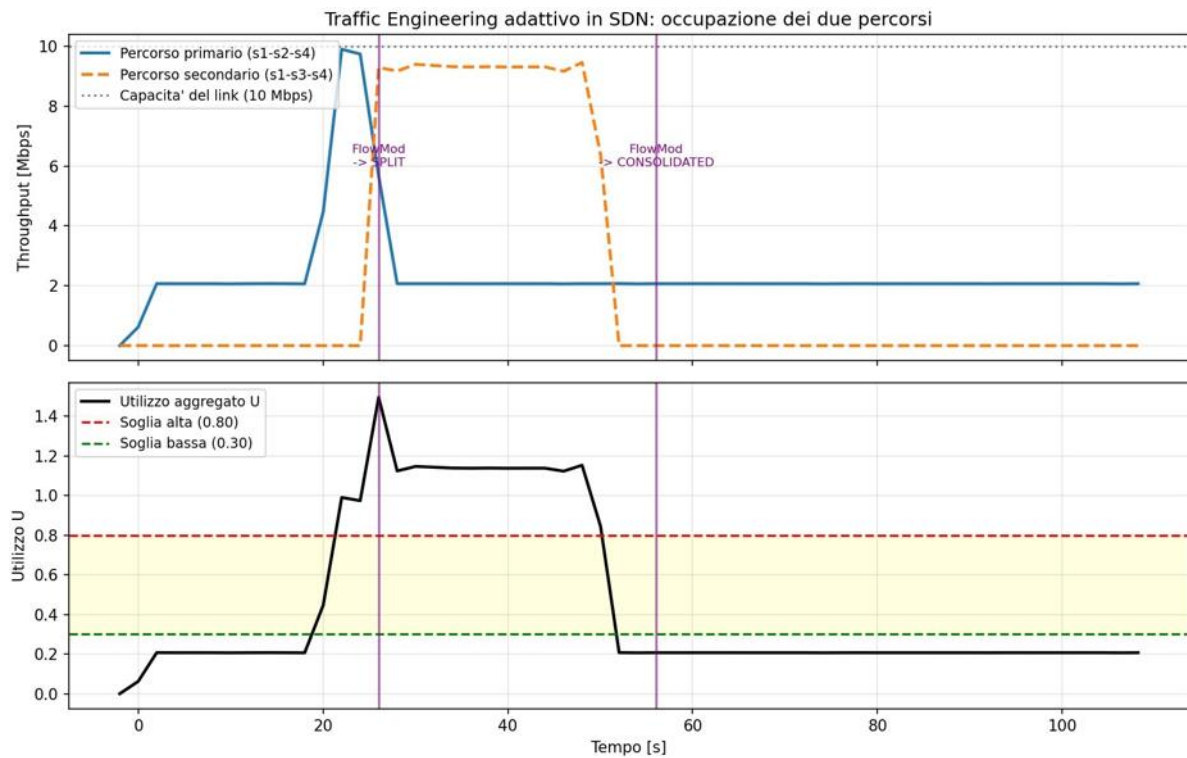


Figura 2 - Traffic Engineering attivo. Le linee verticali indicano gli istanti di invio dei messaggi FlowMod. La fascia gialla è la banda morta fra le due soglie.

6. Discussione critica

6.1 Grafici

Nel campione successivo al rerouting l'utilizzo aggregato raggiunge il **149,6%**, valore apparentemente incompatibile con il traffico immesso in rete. Non si tratta di un errore: l'intervallo di due secondi è a cavallo dello spostamento, cosicché il flusso elastico viene contabilizzato in parte sulla porta del percorso primario e in parte su quella del secondario. L'utilizzo di regime nello stato di split è pari a circa 112%. Un periodo di polling più breve lo attenuerebbe, al costo di un maggiore overhead sul canale di controllo.

Analogamente, il log di iperf3 riporta 320 datagrammi persi nell'intervallo dello spostamento e -320 in quello successivo. I due valori si compensano esattamente e il conteggio finale è pari a zero: quei pacchetti non erano persi ma ancora in transito sul percorso precedente, e sono stati contabilizzati come persi dal ricevitore prima di arrivare.

6.2 Effetti collaterali del rerouting

I pacchetti già oltre lo switch di ingresso al momento della riconfigurazione proseguono sul vecchio percorso, mentre i successivi seguono il nuovo. Poiché i due percorsi hanno ritardi propri, per un breve transitorio i pacchetti possono giungere **fuori ordine**. Nello scenario con Traffic Engineering se ne contano 594, contro 1 nello scenario di riferimento, con un picco di jitter di 484 ms in un singolo intervallo. Con traffico UDP l'effetto è trascurabile; con TCP il riordino verrebbe interpretato come perdita, innescando ACK duplicati e una riduzione della finestra di congestione. Un sistema destinato a traffico TCP dovrebbe quindi limitare la frequenza degli spostamenti.

6.3 Limiti dell'implementazione e possibili estensioni

- I percorsi sono **predefiniti** e non calcolati: il controller sceglie fra due alternative note. Un'estensione naturale è il calcolo dinamico del cammino con un algoritmo tipo SPF applicato al grafo globale della topologia.
- Il flusso da spostare è **fissato a priori**. Una politica più raffinata userebbe le statistiche per flusso per individuare dinamicamente il flusso più oneroso.
- La decisione è **locale al singolo collegamento** e non ottimizza globalmente l'allocazione.
- Il controller è un **single point of failure**: uno scenario realistico richiederebbe un cluster di controller con stato replicato.

7. Esecuzione

Ambiente di riferimento: Ubuntu 22.04 (Python 3.10), Mininet 2.3, Open vSwitch, Ryu 4.34, iperf3, matplotlib. Sono richiesti due terminali; l'ordine dei comandi è vincolante, poiché `mn -c` termina fra gli altri anche il processo `ryu-manager`.

```
# terminale 2 - pulizia di eventuali sessioni  
precedenti sudo mn -c  
  
# terminale 1 - avvio del controller  
ryu-manager controller.py  
  
# terminale 2 - avvio della rete e dello scenario  
sperimentale sudo python3 topology.py --demo  
  
# al termine (circa 115 s): 'exit' nella CLI Mininet  
python3 plot.py /tmp/te_monitor.csv risultati.png
```

Per riprodurre lo scenario di riferimento è sufficiente impostare `THRESHOLD_HIGH = 99.0` in `controller.py` e ripetere la procedura. Lo stato del data plane è ispezionabile in qualunque momento con `sudo ovs-ofctl -O OpenFlow13 dump-flows s1` su un terzo terminale.